

balaji-creator / QRdecomposition

Q

+

<> Code

🔗 Pull requests

🔄 Agents

🎬 Actions

📁 Projects

📖 Wiki

🛡 Security and quality

📊 Insights

👁 Watch

0

🔗

☆

▼

QR Decomposition

📄 BSD-3-Clause license

☆ 0 stars

🔗 2.2k forks

👁 0 watching

🔗 Branches

📈 Activity

🏷 Tags

🌐 Public repository · Forked from [etjabajasphin/QRdecomposition](#)

🔗

🔗 1 Branch

🏷 0 Tags

🔗

🏷

🔍 Go to file

T

Go to file

Add file

↕

Code

...

This branch is **1 commit ahead of** [etjabajasphin/QRdecomposition:main](#) .

🔗 Contribute

▼

🔄 Sync fork

▼

balaji-creator

succes

9db3d59 · now

📄 LICENSE	Initial commit	5 years ago
📄 README.md	succes	now
📄 Screenshot 2026-05-24 165...	succes	now
📄 eq1.jpg	added experiment	5 years ago
📄 eq2.jpg	added experiment	5 years ago
📄 eq3.jpg	added experiment	5 years ago
📄 ex1.jpg	updated	5 years ago
📄 ex2.jpg	updated	5 years ago
📄 ex3.jpg	updated	5 years ago
📄 ex4.jpg	updated	5 years ago
📄 ex6.jpg	updated	5 years ago
📄 input.jpg	added experiment	5 years ago

📖 README

📄 License

✎

☰

Algorithm for QR Decomposition

Aim:

To implement QR decomposition algorithm using the Gram-Schmidt method.

Equipment's required:

1. Hardware – PCs
2. Anaconda – Python 3.7 Installation / Moodle-Code Runner

Algorithm:

1. Intialize the matrix Q and u
2. The vector u and e is given by

$$u_1 = a_1$$

$$u_2 = a_2 - (a_2 \cdot e_1)e_1$$

$$u_3 = a_3 - (a_3 \cdot e_1)e_1 - (a_3 \cdot e_2)e_2$$

$$e_1 = \frac{u_1}{\|u_1\|}$$

$$e_2 = \frac{u_2}{\|u_2\|}$$

3. Obtain the Q matrix

$$Q = (e_1 | e_2 | \dots \dots e_n)$$

$$R = \begin{bmatrix} a_1 \cdot e_1 & a_2 \cdot e_1 & a_3 \cdot e_1 \\ 0 & a_2 \cdot e_2 & a_3 \cdot e_2 \\ 0 & 0 & a_3 \cdot e_3 \end{bmatrix}$$

4. Construct the upper triangular matrix R

Program:

Gram-Schmidt Method

```
'''
Program to QR decomposition using the Gram-Schmidt method
Developed by:Balaji B
RegisterNumber: 212225040040
'''

import os
os.environ["OPENBLAS_NUM_THREADS"]="1"
import numpy as np
def QR_Decomposition(A):
    n,m= A.shape
    q=np.empty((n,n))
    u=np.empty((n,n))
    u[:,0]=A[:,0]
    q[:,0]=u[:,0] / np.linalg.norm(u[:,0])
```



```

for i in range(1,n):
    u[:,i]=A[:,i]
    for j in range(i):
        u[:,i]-(A[:,i]@q[:,j]*q[:,j])
        q[:,i]=u[:,i] / np.linalg.norm(u[:,i])
r=np.zeros((n,m))
for i in range(n):
    for j in range(i,m):
        r[i,j] = A[:,j]@q[:,i]
    print(f"The Q Matrix is \n {q}")
    print(f"The R Matrix is \n {r}")
a = np.array(eval(input()))
QR_Decomposition(a)

```

Output

```

1 '''
2 Program to QR decomposition using the Gram-Schmidt method
3 Developed by:Balaji B
4 RegisterNumber: 212225040040
5 '''
6 import os
7 os.environ["OPENBLAS_NUM_THREADS"]="1"
8 import numpy as np
9 def QR_Decomposition(A):
10     n,m= A.shape
11     q=np.empty((n,n))
12     u=np.empty((n,n))
13     u[:,0]=A[:,0]
14     q[:,0]=u[:,0] / np.linalg.norm(u[:,0])
15     for i in range(1,n):
16         u[:,i]=A[:,i]
17         for j in range(i):
18             u[:,i]-(A[:,i]@q[:,j]*q[:,j])
19             q[:,i]=u[:,i] / np.linalg.norm(u[:,i])
20     r=np.zeros((n,m))
21     for i in range(n):
22         for j in range(i,m):
23             r[i,j] = A[:,j]@q[:,i]
24     print(f"The Q Matrix is \n {q}")
25     print(f"The R Matrix is \n {r}")
26     a = np.array(eval(input()))
27     QR_Decomposition(a)

```

Input	Expected	Got	
✓ ([[1, 1, 0], [1,0,1], [0, 1, 1]])	The Q Matrix is [[0.70710678 0.40824829 -0.57735027] [0.70710678 -0.40824829 0.57735027] [0. 0.81649658 0.57735027]] The R Matrix is [[1.41421356 0.70710678 0.70710678] [0. 1.22474487 0.40824829] [0. 0. 1.15470054]]	The Q Matrix is [[0.70710678 0.40824829 -0.57735027] [0.70710678 -0.40824829 0.57735027] [0. 0.81649658 0.57735027]] The R Matrix is [[1.41421356 0.70710678 0.70710678] [0. 1.22474487 0.40824829] [0. 0. 1.15470054]]	✓
✓ ([[12, -51, 4], [6, 167, -68], [-4, 24, -41]])	The Q Matrix is [[0.85714286 -0.39428571 -0.33142857] [0.42857143 0.98285714 0.03428571] [-0.28571429 0.17142857 -0.94285714]] The R Matrix is [[14. 21. -14.] [0. 175. -70.] [0. 0. 35.]]	The Q Matrix is [[0.85714286 -0.39428571 -0.33142857] [0.42857143 0.98285714 0.03428571] [-0.28571429 0.17142857 -0.94285714]] The R Matrix is [[14. 21. -14.] [0. 175. -70.] [0. 0. 35.]]	✓

Passed all tests! ✓

Result

Releases

No releases published

[Create a new release](#)

Packages

No packages published

[Publish your first package](#)

Contributors

No contributors